

华东师范大学数据学院上机实践报告

课程名称： 操作系统

年级： 2022 级

上机实践成绩：

指导教师： 翁楚良

姓名：

上机实践名称： 实验三-页表

学号：

上机实践日期： 2024.5.27

一、实验目的

学习页表的实现机制，用户拷贝数据到内核态的方法。

二、实验内容

作业一

在 xv6 中，如果用户态调用系统调用，就会切换到内核态，这中间一定是有开销的，至少 CPU 要保存用户态进程的上下文，然后 CPU 被内核占有，系统调用完成后再切换回来。作业一本质是加速 `getpid()`，思路是为每一个进程多分配一个虚拟地址位于 `USYSCALL` 的页，然后这个页的开头保存一个 `usyscall` 结构体，结构体中存放这个进程的 `pid`。

作业二

递归遍历页表，碰到有效的就遍历进下一层页表，打印页表内容。

定义一个名为 `vmprint()` 的函数。它应该使用 `pagetable_t` 参数，并以下面描述的格式打印该页表。

```
page table 0x0000000087f6e000
..0: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..0: pte 0x0000000021fdac1f pa 0x0000000087f6b000
.. .. ..1: pte 0x0000000021fda00f pa 0x0000000087f68000
.. .. ..2: pte 0x0000000021fd9c1f pa 0x0000000087f67000
..255: pte 0x0000000021fdb401 pa 0x0000000087f6d000
.. ..511: pte 0x0000000021fdb001 pa 0x0000000087f6c000
.. .. ..509: pte 0x0000000021fdd813 pa 0x0000000087f76000
.. .. ..510: pte 0x0000000021fddc07 pa 0x0000000087f77000
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000
```

在 `exec.c` 中 `return argc` 之前插入 `if(p->pid==1) vmprint(p->pagetable)`，以打印第一个进程的页表，并通过 `make Grade` 的 PTE 打印输出测试。

三、使用环境

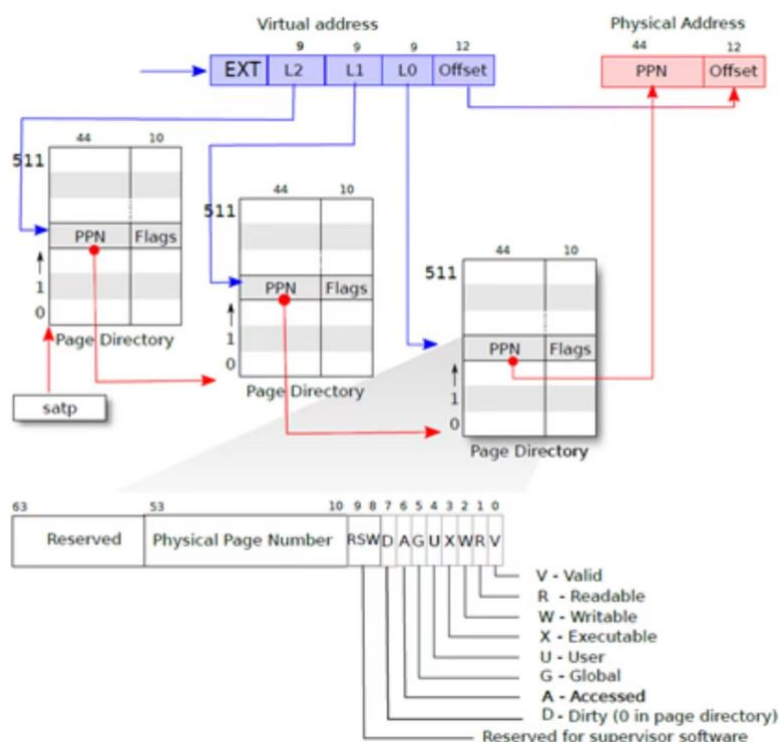
VMware Workstation Pro

Visual Studio Code

四、实验过程及结果

简单的虚拟地址只有一级页表，这一级页表中包括了所有能访问到的物理地址页。但是由于这样的结构太直接，导致了这唯一的一级页表显得十分庞大，如果给一个普通的进程分配这么大的页表会导致有些根本用不上的 PTE(Page Table Entry)会浪费掉。

于是为了能够节省空间，必然会牺牲一定的时间作为代价。因此，多级页表为了解决单级页表的空间分配太大而产生。在页表查询过程中每一个中间级别的页表的 PTE 都包含了下一个级别的页表的物理地址信息。从而能够一级一级的索引到最末端的 leafPTE。



用户程序和内核程序中的机器指令使用虚拟地址，而访问 RAM 或物理内存用的是物理地址。在 xv6 中通过页表硬件将每个虚拟地址映射到物理地址的方法来保证程序的正确执行。

页表以三级树的形式存储在物理内存中。树的根是一个 4KB 的页表，包含 512 个 PTE (Page Table Entry)，这些页表项包含树的下一级页表的物理地址。这些页面中的每一页都包含树中下一级的 512 个 PTE。分页硬件使用 27 位中的高 9 位在根页表中选择一个 PTE，中间的 9 位用于在树的下一级的页表中选择一个 PTE，并使用后 9 位来选择最终的 PTE。

如果转换地址所需的三个 PTE 中的任何一个不存在，则分页硬件将引发错误。每

个 PTE 都包含一些标志位，告诉分页硬件如何允许使用关联的虚拟地址。PTE_V 表示 PTE 是否存在：如果未设置，对页面的引用会导致错误。PTE_R 控制是否允许指令读取页。PTE_W 控制是否允许指令写入页。PTE_X 控制 CPU 是否可以将页面内容解释为指令并执行它们。PTE_U 控制是否允许用户模式下的指令访问页面；如果未设置 PTE_U，则 PTE 只能在管理员模式下使用。标志和所有其他页面硬件相关结构在（kernel/riscv.h）中定义。

Task1:

当创建一个进程时，将一个只读页面映射到虚拟地址 USYSCALL 所在页面。在该页面的开头，存储了一个 struct usyscall 结构变量，初始化该变量，使其存储当前进程的 PID。创建每个进程时，在 USYSCALL 处映射一个只读页面（在 memlayout.h 中定义的 VA）。在此页面的开头，存储一个 struct usyscall（也在 memlayout.h 中定义），并初始化它以存储当前进程的 PID。

在 kernel/proc.h 中新增一个域，存储共享内存块的物理地址。

```

106     struct inode *cwd;           // Current directory
107     char name[16];              // Process name (debugging)
108     struct usyscall *usyscallpage; //shared with kernel
109 };

```

在 kernel/proc.c 中的 allocproc 函数中增加申请共享申请页。

```

123     // Allocate a trapframe page.
124     if((p->trapframe = (struct trapframe *)kalloc()) == 0){
125         freeproc(p);
126         release(&p->lock);
127         return 0;
128     }
129
130     //Allocate a share page
131     if((p->usyscallpage = (struct usyscall *)kalloc()) == 0){
132         freeproc(p);
133         release(&p->lock);
134         return 0;
135     }
136     p->usyscallpage->pid = p->pid;
137
138     // An empty user page table.
139     p->pagetable = proc_pagetable(p);
140     if(p->pagetable == 0){
141         freeproc(p);
142         release(&p->lock);
143         return 0;
144     }

```

在 kernel/proc.c 的 proc_pagetable 函数中增加在内核中共享内存页的初始化，以及对共享内存块的页表初始化。在用户进程页表中将 usyscall 映射到 USYSCALL，这样在访问 USYSCALL 的时候就相当于访问了用户进程结构体的 usyscall 成员。整个配置用户进程页表的过程中都是在内核态进行，所以整个 proc 结构体的地址信息都是处于内核页表的映射下的，而内核页表的这部分是直接映射，所以相当于物理地址了。仿照 trapframe 在 proc_pagetable() 中添加映射，这时候页面的权限需要 PTE_R 和 PTE_U。

mappages 将虚拟地址页号到物理地址页号的映射安装到页表中。在 size 范围内的虚拟地址将以页大小为间隔分别执行此操作。对于要映射的每个虚拟地址，mappages 调用 walk 来查找该地址的 PTE 地址，然后初始化 PTE 以保存相关的物理页码和所需的权限（PTE_W、PTE_X 和/或 PTE_R），并将 PTE 的标志位 PTE_V 设置为有效。

```

182 pagetable_t
183 proc_pagetable(struct proc *p)
184 {
185     pagetable_t pagetable;
186
187     // An empty page table.
188     pagetable = uvmcreate();
189     if(pagetable == 0)
190         return 0;
191
192     // map the trampoline code (for system call return)
193     // at the highest user virtual address.
194     // only the supervisor uses it, on the way
195     // to/from user space, so not PTE_U.
196     if(mappages(pagetable, TRAMPOLINE, PGSIZE,
197                (uint64)trampoline, PTE_R | PTE_X) < 0){
198         uvmfree(pagetable, 0);
199         return 0;
200     }
201
202     // map the trapframe just below TRAMPOLINE, for trampoline.S.
203     if(mappages(pagetable, TRAPFRAME, PGSIZE,
204                (uint64)(p->trapframe), PTE_R | PTE_W) < 0){
205         uvmunmap(pagetable, TRAMPOLINE, 1, 0);
206         uvmfree(pagetable, 0);
207         return 0;
208     }
209
210     if(mappages(pagetable, USYSCALL, PGSIZE, (uint64)(p->usyscallpage), PTE_R|PTE_U) < 0){
211         uvmunmap(pagetable, TRAMPOLINE, 1, 0);
212         uvmunmap(pagetable, TRAPFRAME, 1, 0);
213         uvmfree(pagetable, 0);
214         return 0;
215     }
216     return pagetable;
217 }

```

在 kernel/proc.c 的 freeproc 函数中增加释放共享内存块。

```

158 static void
159 freeproc(struct proc *p)
160 {
161     if(p->trapframe)
162         kfree((void*)p->trapframe);
163     p->trapframe = 0;
164     if(p->usyscallpage) //增加释放共享内存块
165         kfree((void*)p->usyscallpage);
166     p->usyscallpage = 0;
167     if(p->pagetable)
168         proc_freepagetable(p->pagetable, p->sz);
169     p->pagetable = 0;

```

在 kernel/proc.c 的 proc_freepagetable 函数中增加一行释放页表中共享内存页项。

```

219 // Free a process's page table, and free the
220 // physical memory it refers to.
221 void
222 proc_freepagetable(pagetable_t pagetable, uint64 sz)
223 {
224     uvmunmap(pagetable, TRAMPOLINE, 1, 0);
225     uvmunmap(pagetable, TRAPFRAME, 1, 0);
226     uvmunmap(pagetable, USYSCALL, 1, 0);
227     uvmfree(pagetable, sz);
228 }

```

通过 pgtbltest 时 ugetpid 测试用例显示 OK。

```

make: "kernel/kernel"已是最新。
== Test pgtbltest == (5.9s)
== Test   pgtbltest: ugetpid ==
    pgtbltest: ugetpid: OK

```

Task2:

通过作业二的提示，将 vmprint()放在 kernel/vm.c 中。本实验实现一个 vmprint()函数来遍历页表并打印，在 xv6 有 freewalk 函数，实现释放页表内存，模仿该函数来写 vmprint()函数。

递归释放页表，freewalk()函数如下：

```

267 void
268 freewalk(pagetable_t pagetable)
269 {
270     // there are 2^9 = 512 PTEs in a page table.
271     for(int i = 0; i < 512; i++){
272         pte_t pte = pagetable[i];
273         if((pte & PTE_V) && (pte & (PTE_R|PTE_W|PTE_X)) == 0){
274             // this PTE points to a lower-level page table.
275             uint64 child = PTE2PA(pte);
276             freewalk((pagetable_t)child);
277             pagetable[i] = 0;
278         } else if(pte & PTE_V){
279             panic("freewalk: leaf");
280         }
281     }
282     kfree((void*)pagetable);
283 }

```

模仿 freewalk() 函数，编写 vmprint() 函数和 vmprint_helper() 函数。打印页表的格式中每行前面有缩进表示页表深度，创建 indent[] 数组输出其字符串。页表深度为 1, 2, 3，做传参判断以示错误。接下来通过 for 循环遍历每个页表项并打印其 pte 和 pa。如果该页表项有效则寻找 child，下一级页表深度加 1。

```

437 void vmprint_helper(pagetable_t pagetable, int depth){
438     static char* indent[] = {
439         "",
440         "...",
441         "... ..",
442         "... . . ."
443     };
444     if(depth < 1 || depth > 3){ //传参判断
445         panic("vmprint_helper:depth error");
446     }
447     for(int i = 0; i < 512; i++){ //每个页表有512项
448         pte_t pte = pagetable[i];
449         printf("%s%d: pte %p pa %p\n", indent[depth], i, pte, PTE2PA(pte));
450         if((pte & PTE_V) && (pte & (PTE_R|PTE_W|PTE_X)) == 0){ //该页表项有效
451             uint64 child = PTE2PA(pte); //虚拟地址转换为物理地址
452             vmprint_helper((pagetable_t)child, depth+1); //下一级页表，深度加1
453         }
454     }
455 }
456
457 void vmprint(pagetable_t pagetable){
458     printf("page table %p\n", pagetable);
459     vmprint_helper(pagetable, 1);
460 }

```

在 kernel/defs.h 中定义 vmprint 的模型，以便从 exec.c 调用它。

```

170 int          copyout(pagetable_t, uint64, char *, uint64);
171 int          copyin(pagetable_t, char *, uint64, uint64);
172 int          copyinstr(pagetable_t, char *, uint64, uint64);
173 void         vmprint(pagetable_t);
174 // plic.c

```

在 exec.c 中返回的 argc 之前插入 if(p->pid==1) vmprint(p->pagetable)，以输出第一个进程的页表。

```
119     if(p->pid == 1){
120         vimprint(p->pagetable);
121     }
122     return argc; // this ends up in a0, the first argument to main(argc, argv)
```

make qemu 后得到结果显示 OK。

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
page table 0x0000000087f6e000
..0: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..0: pte 0x0000000021fdac1f pa 0x0000000087f6b000
.. .. ..1: pte 0x0000000021fda00f pa 0x0000000087f68000
.. .. ..2: pte 0x0000000021fd9c1f pa 0x0000000087f67000
```

```
make: "kernel/kernel"已是最新。
== Test pte printout == pte printout: OK (6.0s)
```

五、总结

页表是操作系统控制内存含义的机制，它允许 xv6 把一块物理内存映射到不同进程的地址空间，并保护进程各自的内存。页表逻辑上是一个页表入口（PTE）的数组，每个 PTE 包含物理页号码（PPN）和一些标志位。本次实验通过为每一个进程多分配一个虚拟地址位于 USYSCALL 的页，在其开头保存进程 pid 的结构体来加速 getpid()与递归遍历打印页表两项任务，更进一步理解了操作系统中页表的实现机制，用户拷贝数据到内核态的方法。